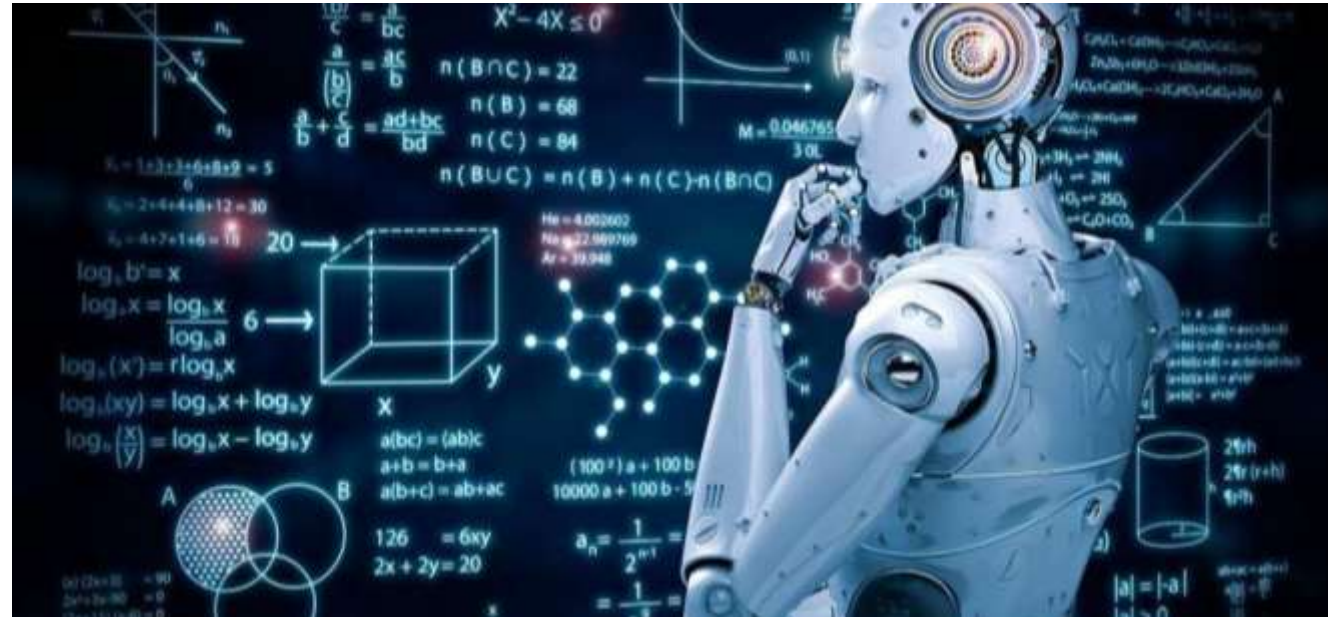




› AI FOR BUSINESS APPLICATIONS

SUPPLEMENTARY READING FOR THE COURSE

HILBERT, S., KRAUS, E., & LINDL, A. (2025). *Machine Learning: Eine Einführung für Psychologie, Geistes- und Sozialwissenschaften* (Quantitative Sozialforschung). Wiesbaden: Springer Fachmedien Wiesbaden.

KOCH, D., KOHNE, A., & BRECHBÜHLER, N. (2025). *Prompt Engineering im Unternehmen – eine Einführung*. Wiesbaden: Springer Gabler

KOCH, D., KOHNE, A., & BRECHBÜHLER, N. (2025). *Prompt Engineering in the Enterprise – An Introduction*. Wiesbaden: Springer Gabler

Kubat, M. (2021). *An Introduction to Machine Learning*. Springer Cham.

-
- > No open book
 - > But: cheat sheet (1 sheet, double-sided, written by yourself)

-
1. Introduction
 2. Prompt Engineering
 3. Machine Learning (Supervised, Unsupervised, Reinforcement Learning)
 4. Implementing AI solutions in companies

› 1. INTRODUCTION

“Artificial intelligence is the science and engineering of making intelligent machines, especially intelligent computer programs.”

(McCarthy, J., Minsky, M., Rochester, N., & Shannon, C. (1955). *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*)

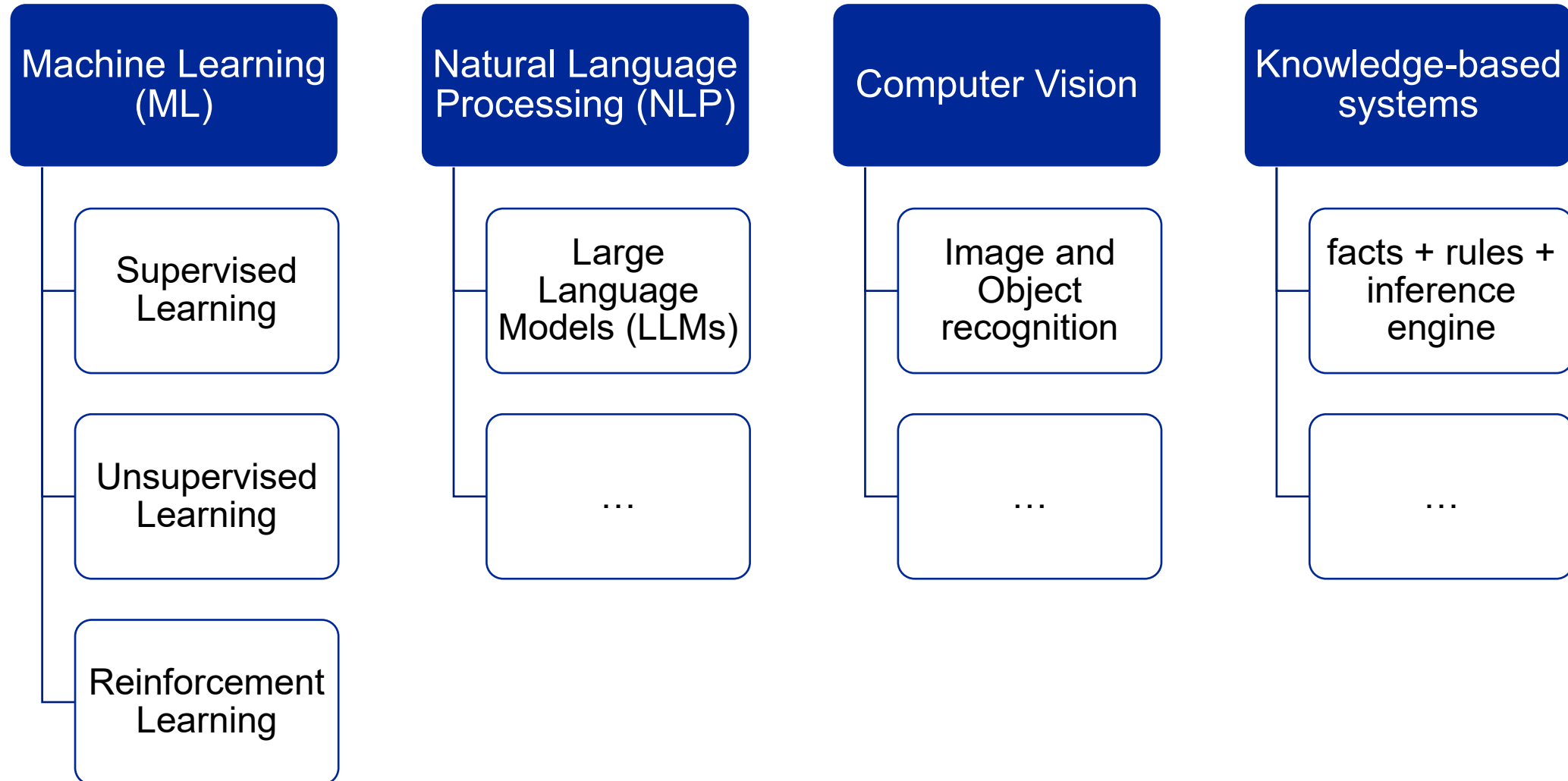
“Artificial intelligence is the ability of a machine to imitate human abilities such as logical reasoning, learning, planning, and creativity.”

(<https://www.europarl.europa.eu/topics/de/article/20200827STO85804/was-ist-kunstliche-intelligenz-und-wie-wird-sie-genutzt>)

“AI is the development of agents that perceive their environment and take actions to achieve their goals.”

(Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*.)

ARTIFICIAL INTELLIGENCE: SOME SUBDOMAINS



Python:



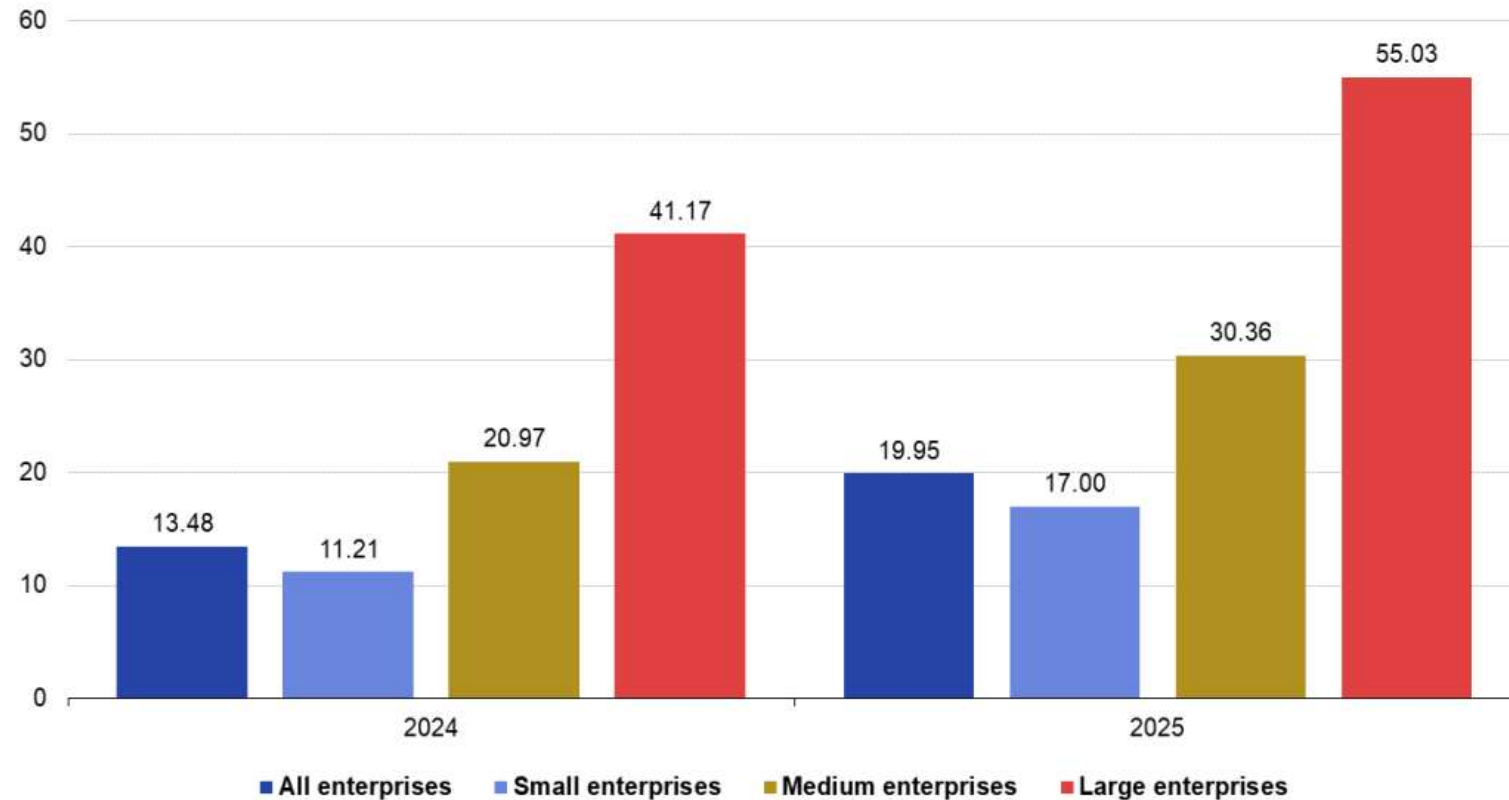
- > by far the most common language for building and training AI models.
- > Major ecosystems: PyTorch, TensorFlow, JAX, and Hugging Face Transformers

Interpreters:

- > <https://www.anaconda.com/download> [free]
- > <https://www.jetbrains.com/de-de/pycharm/> [free for students, recommended]

ARTIFICIAL INTELLIGENCE: BUSINESS IMPACT

Enterprises using AI technologies by size class, EU, 2024 and 2025
(% of enterprises)

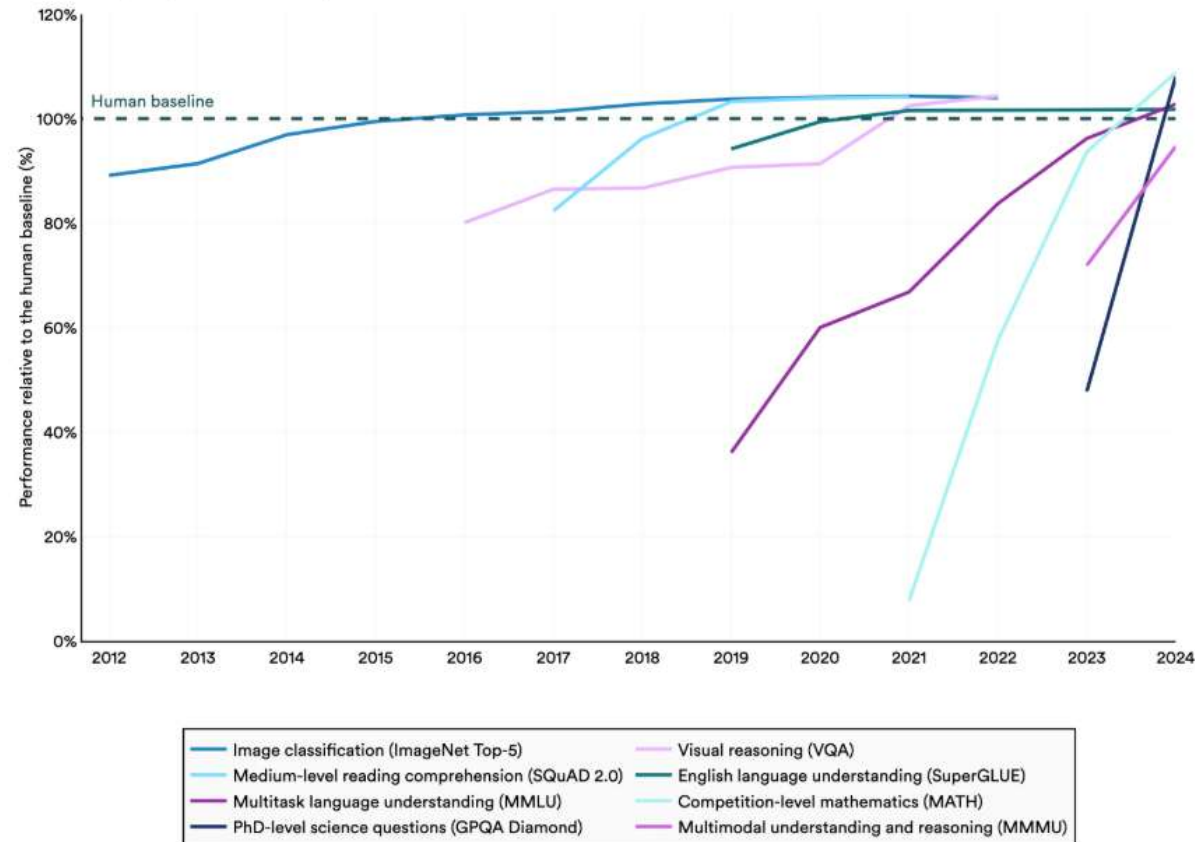


Source: Eurostat (online data code: isoc_eb_ai)

ARTIFICIAL INTELLIGENCE: PERFORMANCE

Select AI Index technical performance benchmarks vs. human performance

Source: AI Index, 2025 | Chart: 2025 AI Index report



Source: Uni Stanford, <https://hai.stanford.edu/ai-index/2025-ai-index-report>

› 2. PROMPT ENGINEERING

PROMPT ENGINEERING: WHAT IS GEN-AI?

GenAI (generative AI) is a type of artificial intelligence that

- > creates new content rather than just analyzing existing data

Examples:

- > DALL-E (text-to-image)

Link: <https://openai.com/de-DE/index/dall-e-3/>

- > ChatGPT (text-to-text)

Link: <https://chatgpt.com/>

- > Suno (text-to-music)

Link: https://suno.com/home?utm_source=chatgpt.com

PROMPT ENGINEERING: WHAT ARE LLMS?

Large Language Models (LLMs):

- > a specialized type of generative AI
- > large neural networks trained on vast amounts of text, with billions of parameters
- > designed to respond to prompts, such as questions or instructions
- > able to generate and continue text, answer questions, translate, summarize, and assist with programming

PROMPT ENGINEERING: LLM EXAMPLES

Selected Large Language Models (LLMs):

> ChatGPT (OpenAI)

> Claude (Anthropic)

Link: <https://claude.ai/new>

> Gemini (Google)

Link: <https://gemini.google.com/app?hl=de>

> Mistral (Open Source)

Link: <https://mistral.ai/>

> LLaMA (Meta)

Link: <https://www.llama.com/>

PROMPT ENGINEERING: HOW LLMS OPERATE

Main Components LLM:

- > Tokenization
- > Transformer architecture

Paper:

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.

Link: https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

Before a transformer can process text, it must first break the text down into small units.

- > These units are called tokens.
- > Tokens are not necessarily whole words
- > Example (simplified Byte Pair Encoding):
 - Sentence: "Chatbots are helpful."
 - Tokens: ["Chat", "bot", "s", "are", "help", "ful", "."]

Tokenization = transforming natural language into machine-readable units.

PROMPT ENGINEERING: LLM TRANSFORMER ARCHITECTURE

1. Input (Tokens → Vectors)

- > Each token is transformed into a vector, called an embedding.
- > To preserve word order, the model also uses positional encoding.

2. Self-Attention (Attention Mechanism)

- > Self-attention allows the model to evaluate the relationship of each word
- > to all other words in the sentence and to identify the most relevant contextual information.

Example:

In a sentence such as *“The dog chases the cat because it is fast,”* attention helps the model infer which noun the pronoun *“it”* most likely refers to.

3. Feedforward network

- > After the attention layer:
the vectors pass through small neural networks that amplify or suppress certain patterns.

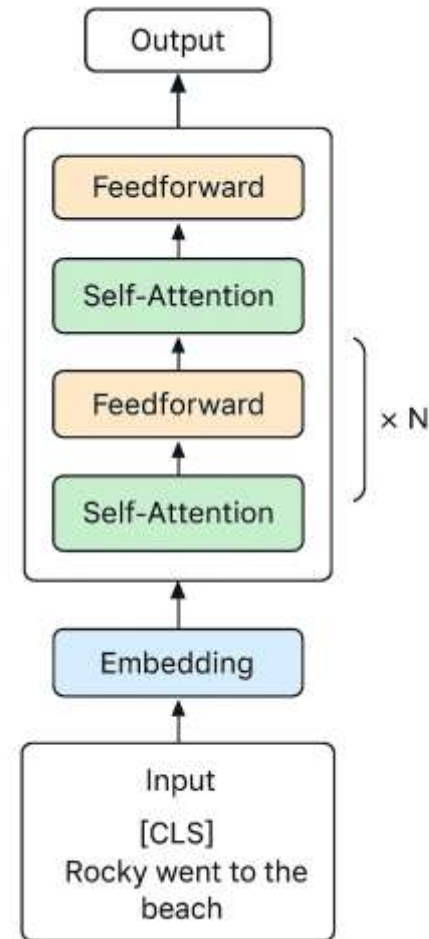
4. Multiple Layers (Stacking)

- > Transformers consist of many such attention and feedforward blocks
- > The greater the number of layers, the more abstract and informative the model's internal representation of language becomes.

5. Output (Decoding)

- > The model calculates which token is most likely to come next.
- > Example: Prompt “The sun rises in the ____.” → Most likely token: “east”

PROMPT ENGINEERING: LLM TRANSFORMER ARCHITECTURE



PROMPT ENGINEERING: DEFINITION

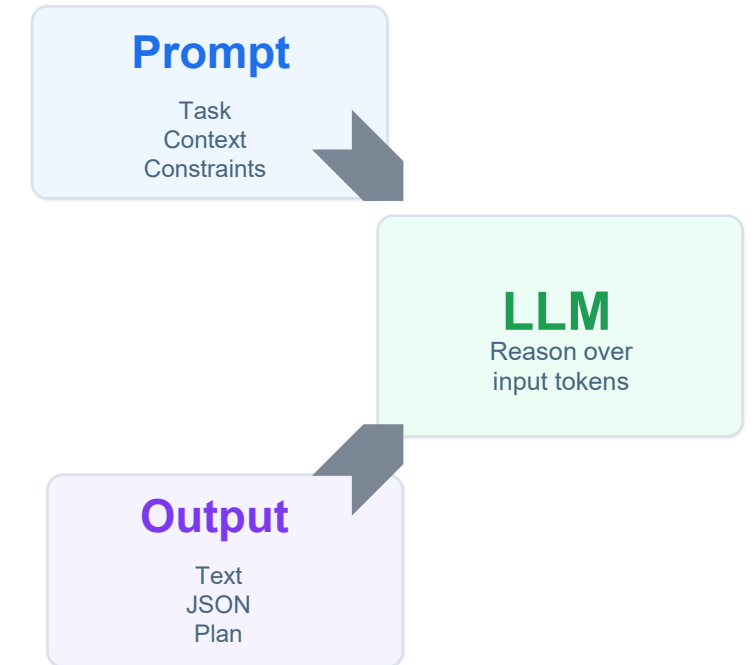
Prompt engineering is the iterative design of instructions, context, examples, and output constraints so that a model produces useful and reliable results for a specific task.

- > It is not only wording; it also includes structure and evaluation.
- > It remains important because model outputs are probabilistic and context-sensitive.
- > Good prompts reduce ambiguity; they do not guarantee truth.

PROMPT ENGINEERING: GOALS

Well-designed prompts reduce ambiguity and improve reliability.

- > Clarify the task for the model
- > Control structure, tone, and scope
- > Support reproducible classroom and lab workflows



PROMPT ENGINEERING: ANATOMY OF AN EFFECTIVE PROMPT

Prompt components

Role / audience

Task

Context / data

Constraints

Output format

Quality criteria

Not every prompt needs all components, but adding them deliberately makes behavior easier to predict and evaluate.

Example

```
Role: You are a statistics tutor.  
Task: Explain logistic regression.  
Audience: First-year economics students.  
Constraints: Use simple language and one example.  
Output: 4 bullet points + 1 caution about misuse.
```

PROMPT ENGINEERING: CORE PROMPT TECHNIQUES

Zero-shot prompting

Give the task directly. Best for simple instructions when the model already knows the pattern.

Few-shot prompting

Provide one or more examples. Useful when you need a specific style, format, or decision boundary.

Task decomposition

Split a complex problem into smaller steps or sub-questions to reduce ambiguity and failure modes.

Grounding / context injection

Attach relevant source text, data, or retrieved passages so the model has the right evidence.

Selection heuristic: Choose the simplest pattern that matches the task



PROMPT ENGINEERING: FROM WEAK TO STRONGER PROMPT

Prompt quality often comes from added structure, not fancy wording

Weak prompt

```
Summarize this article.
```

- Ambiguous task
- No audience
- No structure
- No quality control

Improved prompt

```
You are a teaching assistant.  
Summarize the article for first-year students.  
Use 5 bullet points and one key takeaway.  
Do not introduce facts not present in the text.
```

- Defines audience and goal
- Specifies output structure
- Adds a boundary against hallucination
- More reproducible across runs

PROMPT ENGINEERING IS AN EVALUATION LOOP

The best prompt is discovered through testing, not guessed in one try



Typical evaluation dimensions

Accuracy / faithfulness



Format compliance



Latency and cost



Robustness across cases



Practical rule: keep a small benchmark set of representative prompts and compare revisions systematically.

PROMPT ENGINEERING: PROMPT PATTERNS BY TASK TYPE

Different tasks benefit from different prompt structures

Classification

Label the sentiment as positive, neutral, or negative. Return only the label.

- Explicit label set
- Minimal output

Extraction

Extract the date, sender, and invoice total. Return valid JSON with null for missing fields.

- Schema first
- Handle missing data

Summarization

Summarize in 5 bullet points for first-year students and include one limitation of the study.

- Audience level
- Length control

Tutoring

Teach this concept using one analogy, then ask the student a check question before continuing.

- Interactive steps
- Feedback loop

PROMPT ENGINEERING: KEY TAKEAWAYS

Prompt engineering is useful, but it works best with evaluation and domain knowledge

- Good prompts specify task, audience, context, constraints, and output format.
- Few-shot examples and external context help when the task pattern is not obvious.
- Prompt engineering is an iterative evaluation process, not a one-shot trick.
- Prompts improve reliability, but they do not remove hallucinations or bias.
- For projects, document prompts, test cases, and failure modes.

› 3. MACHINE LEARNING

SUPERVISED LEARNING: DEFINITION

Supervised learning is a machine learning approach in which an algorithm is trained

- > on labeled data
- > to learn a mapping from inputs (features) to known outputs (targets).

SUPERVISED LEARNING: TASK

Task: predict the correct output for new, unseen data.

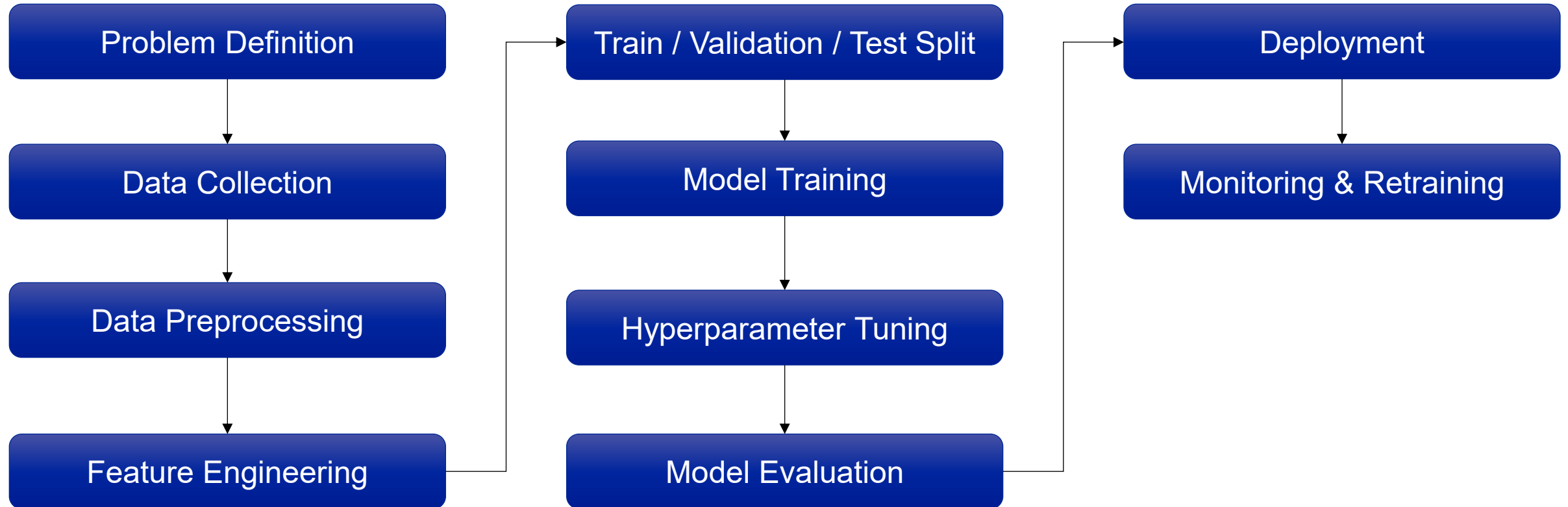
Types:

- > *Classification* → predict a category (e.g., spam vs. not spam)
- > *Regression* → predict a numerical value (e.g., stock prices)

SUPERVISED LEARNING: ALGORITHMS

Linear	Tree-based	Neural Nets	Others
Linear Regression	Decision Trees	ANN	Support Vector Machines
Logistic Regression	Random Forest	Deep Learning	k-Nearest Neighbors
...	Gradient Boosting	...	Naive Bayes
	...		...

SUPERVISED LEARNING: PROCESS

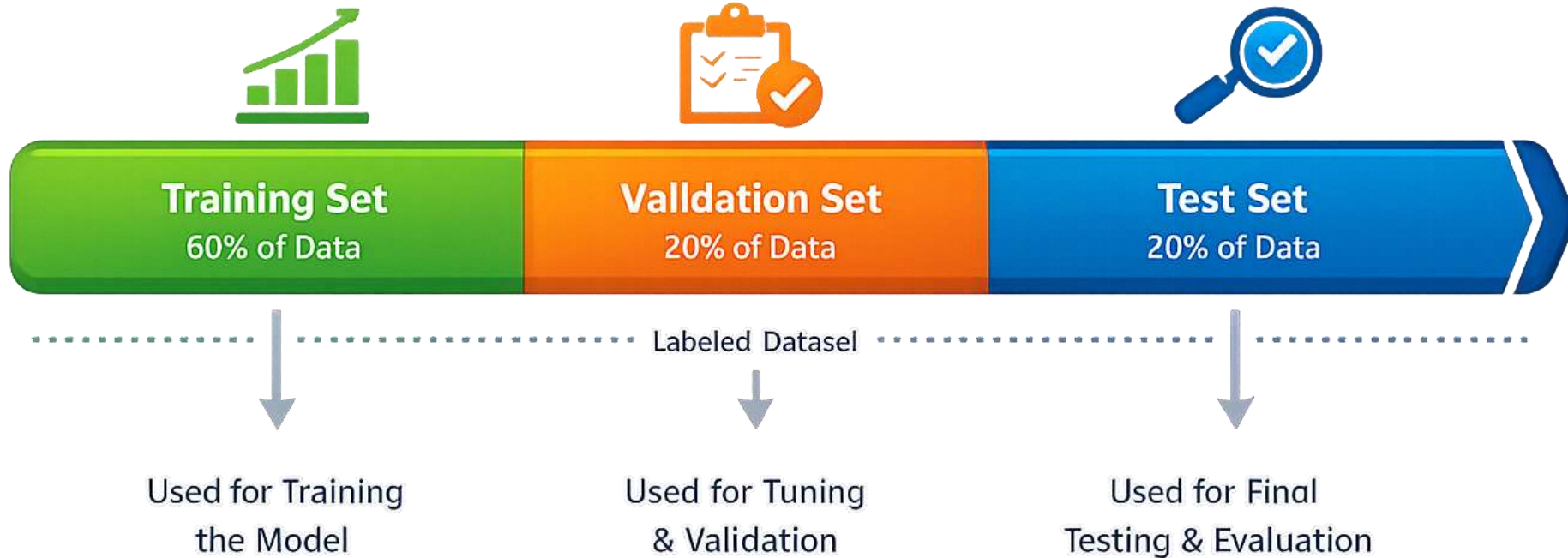


SUPERVISED LEARNING: TRAIN, VALIDATION, TEST SPLIT

Train, Validation, Test Split

Dividing the dataset into three subsets

*Percentages shown are illustrative only



SUPERVISED LEARNING: MODEL TRAINING

A model is trained by minimizing the average loss over the dataset w.r.t. to a loss function:

$$L(y, \hat{y})$$

that quantifies the error between prediction ($\hat{y}$) and true target (y).

SUPERVISED LEARNING: TYPICAL LOSS FUNCTIONS

Classification:

- > Quadratic loss, Log loss / cross entropy, Hinge loss

Regression:

- > Squared loss: $(y - \hat{y})^2$
- > Absolute loss: $|y - \hat{y}|$

SUPERVISED LEARNING: MODEL EVALUATION IN CLASSIFICATION

- > Accuracy – proportion of correctly classified samples
- > Precision – proportion of predicted positives that are actually positive
- > Recall (Sensitivity) – proportion of actual positives correctly identified
- > F1-Score – harmonic mean of precision and recall
- > Specificity – proportion of actual negatives correctly identified
- > ROC-AUC – measures discrimination ability across classification thresholds
- > Log Loss / Cross-Entropy Loss – penalizes incorrect predicted probabilities
- > Confusion Matrix – summarizes true/false positives and negatives

SUPERVISED LEARNING: MODEL EVALUATION IN REGRESSION

- > MAE (Mean Absolute Error) – average absolute difference predicted and true values
- > MSE (Mean Squared Error) – average squared error
- > RMSE (Root Mean Squared Error) – square root of MSE; same unit as target
- > R^2 (Coefficient of Determination) – proportion of variance explained by the model
- > Adjusted R^2 – R^2 adjusted for number of predictors
- > MAPE (Mean Absolute Percentage Error) – average percentage error
- > Explained Variance – measures how much variability is captured by the model

SUPERVISED LEARNING: CLASSIFICATION AND REGRESSION TREES

Algorithm (sketch):

1. Root = entire train data set
2. Search for the feature that yields the most informative split
 - > Classification: Reduction in Gini impurity, entropy
 - > Regression: Reduction in variance or squared error
3. Recursion: repeat 2. for each branch / child until stopping criterion is reached
4. Leaves = final prediction

SUPERVISED LEARNING: CLASSIFICATION AND REGRESSION TREES

Gini impurity and entropy:

let p_i be the relative frequency of category $i \in \{1, \dots, K\}$, then

$$\textit{Gini impurity} = 1 - (p_1^2 + \dots + p_K^2)$$

$$\textit{Entropy} = -(p_1 \log_2 p_1 + \dots + p_K \log_2 p_K)$$

SUPERVISED LEARNING: CLASSIFICATION AND REGRESSION TREES

Decision Tree Computation (in Class)

Ensembles = combination of several models

Types:

1. Voting:

- Majority (hard voting) or averaging (soft voting) of the single predictions

2. Stacking:

- Higher-level meta-model that takes single predictions as inputs

Types:

3. *Bagging (Bootstrap Aggregating)*

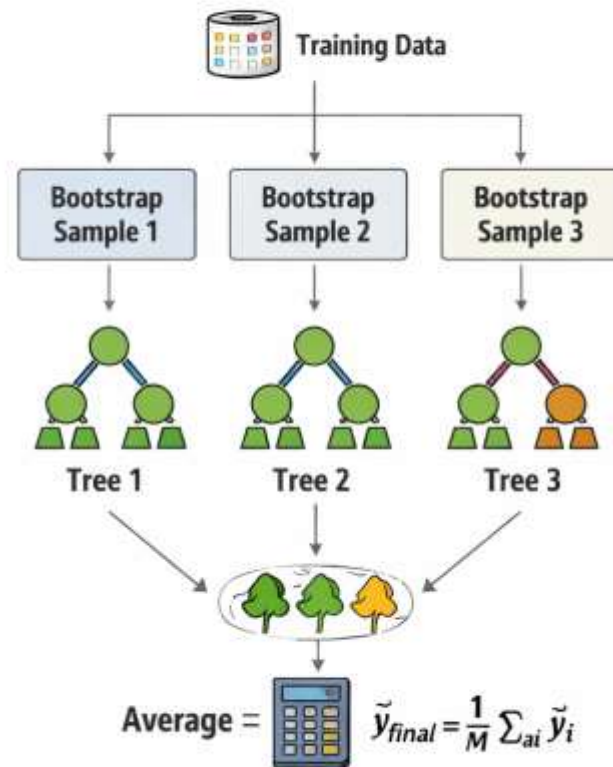
- trains many models independently on different random bootstrap samples
- predictions are combined, usually by averaging or voting,
- Example: **Random Forest**

4. *Boosting*

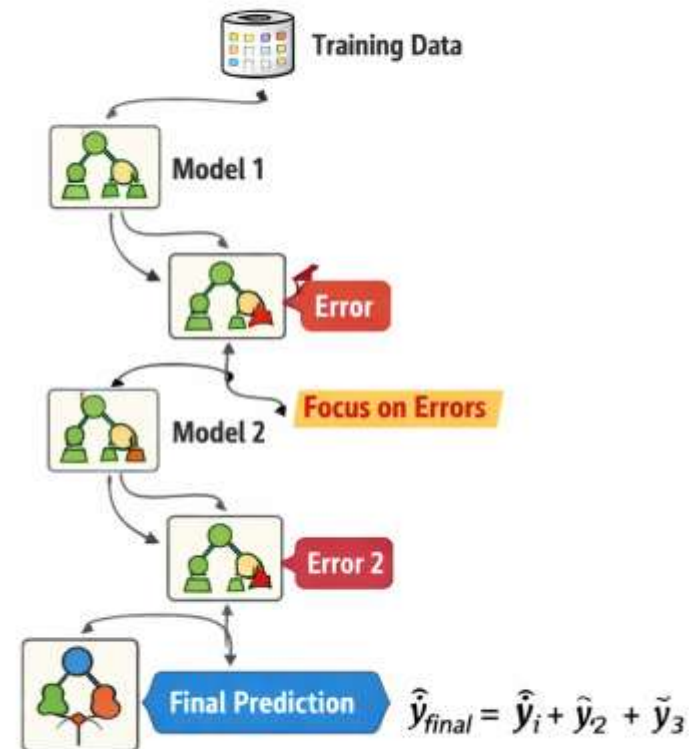
- trains models sequentially,
- each new model focuses on the errors of the previous ones
- Example: **Gradient Boosting**

TREE-BASED ENSEMBLE MODELS: BAGGING AND BOOSTING

Bagging (Random Forest)



Boosting (Gradient Boosting)



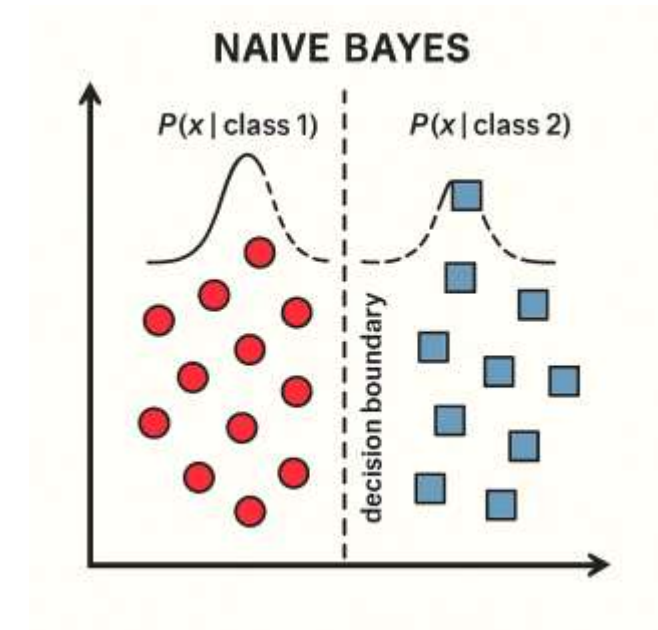
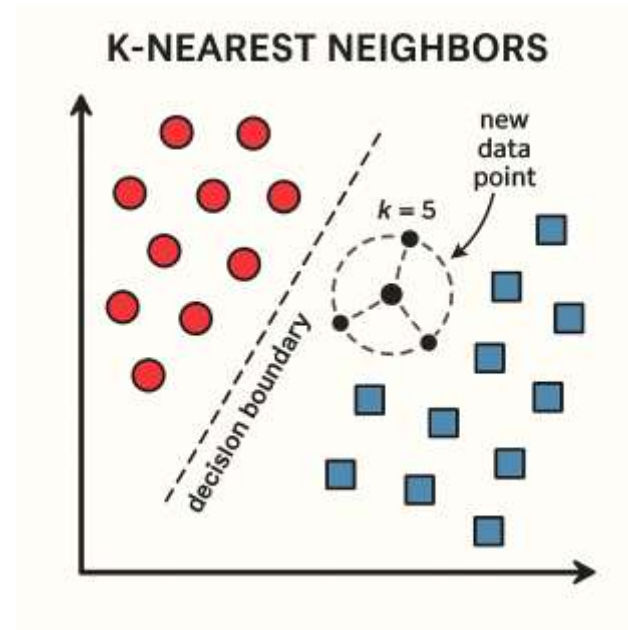
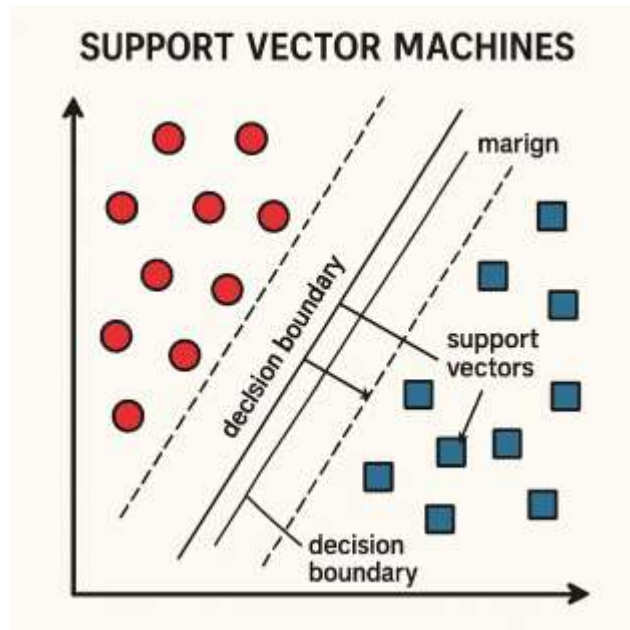
Algorithm (sketch):

1. Initialize: start with simple prediction (e.g., mean)
2. Round r :
 - Compute pseudo-residuals (errors) of prediction in round $r-1$
 - Train new model on pseudo-residuals
 - Add model prediction to previous model with weighting factor (learning rate)
3. Repeat 2. until stopping criterion is reached

SUPERVISED LEARNING: GRADIENT BOOSTING

Gradient Boosting Computation (in Class)

SUPERVISED LEARNING: OTHER ALGORITHMS



UNSUPERVISED LEARNING: DEFINITION

Unsupervised learning is a machine learning approach in which an algorithm is trained

- > on unlabeled data
- > to discover underlying patterns, structures, or groupings without a predefined target.

UNSUPERVISED LEARNING: ALGORITHMS

Clustering	Dimensionality Reduction	Density Estimation	Anomaly Detection
• k-means • Hierarchical clustering • DBSCAN • Gaussian Mixture Models (GMM / EM) ...	• Principal Component Analysis (PCA) • t-SNE • UMAP • Autoencoders • Factor Analysis ...	• Kernel Density Estimation (KDE) • Gaussian Mixture Models • Histograms / Parzen windows ...	• Local Outlier Factor (LOF) • Isolation Forest • One-Class SVM • Autoencoder-based anomaly detection ...

UNSUPERVISED LEARNING: K-MEANS

Algorithm (sketch):

1. Choose the number of clusters k
2. Initialize cluster centers (centroids)
3. Assign each data point to the nearest centroid
4. Recompute the centroids: for cluster C with data $x_1, \dots, x_n$:

$$\mu_C = \frac{1}{n}(x_1 + \dots + x_n)$$

5. Repeat assignment and update steps

UNSUPERVISED LEARNING: K-MEANS

K-Means Computation (in Class)

[OPTIONAL] REINFORCEMENT LEARNING: DEFINITION

Reinforcement learning is a machine learning approach in which an algorithm learns

- > by interacting with an environment and receiving rewards or penalties,
- > with the goal of learning a policy that maximizes cumulative reward over time.



[OPTIONAL] REINFORCEMENT LEARNING: COMPONENTS

- > Agent: The learning system that makes decisions
- > Environment: The world in which the agent operates
- > State (s): The current situation of the environment
- > Action (a): What the agent can do
- > Reward (r): The feedback the agent receives for its actions
- > Policy (π): The strategy that determines which action is chosen in which state

[OPTIONAL] REINFORCEMENT LEARNING: Q-LEARNING

Algorithm (sketch):

Initialization

- Choose the learning rate α , discount factor γ , and exploration rate ϵ .
- Initialize a Q-table for all state-action pairs (s, a) , for example with zeros

1. Choose an action

- With probability ϵ : choose a random action (exploration)
- With probability $1 - \epsilon$: choose the action with the highest Q-value (exploitation)

2. Execute the action: Perform action a and observe the reward r and the new state s'

3. Update Q-values: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

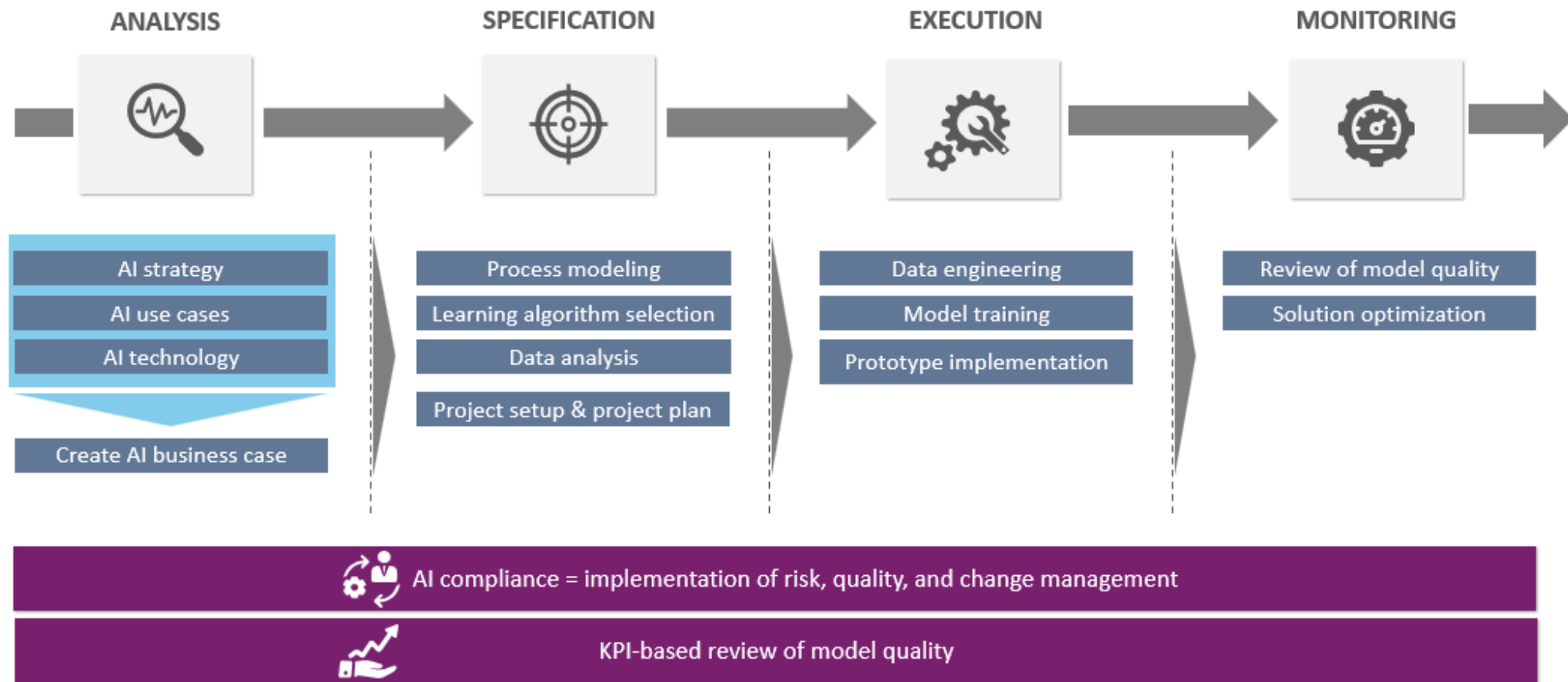
4. Repeat 1. – 3. until stopping criterion

[OPTIONAL] REINFORCEMENT LEARNING: Q-LEARNING

Q-Learning Computation (in Class)

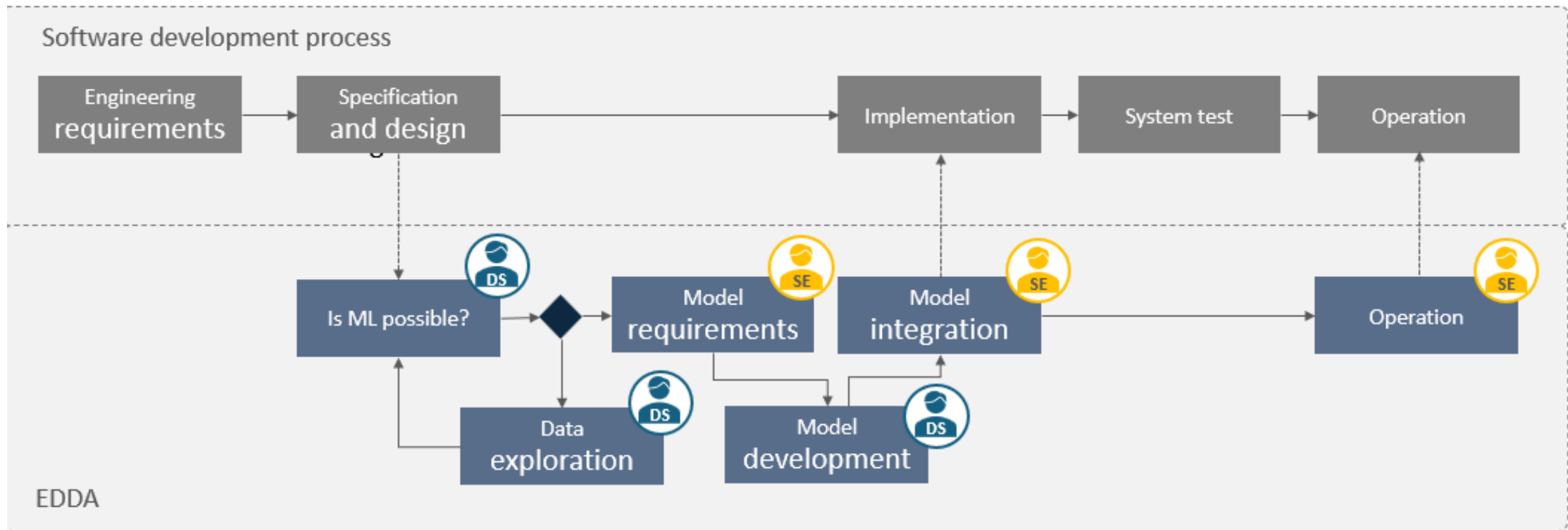
› 4. IMPLEMENTING AI SOLUTIONS IN COMPANIES

IMPLEMENTING AI: PROJECT WORKFLOW



Source: DSC GmbH

THE PATH TO AN AI APPLICATION



Source: EDDA - Engineering Data-Driven Applications

Dr. Marc Hesenius, University of Duisburg-Essen

IMPLEMENTING AI: TASKS IN A DATA SCIENCE PROJECT



DOMAIN EXPERT

Is familiar with the company's business processes and the industry



DATA SCIENTIST

Has AI expertise
Masters techniques of data analysis
Has basic knowledge of software development



DATA ENGINEER

Extracts data from a wide range of systems and makes it available

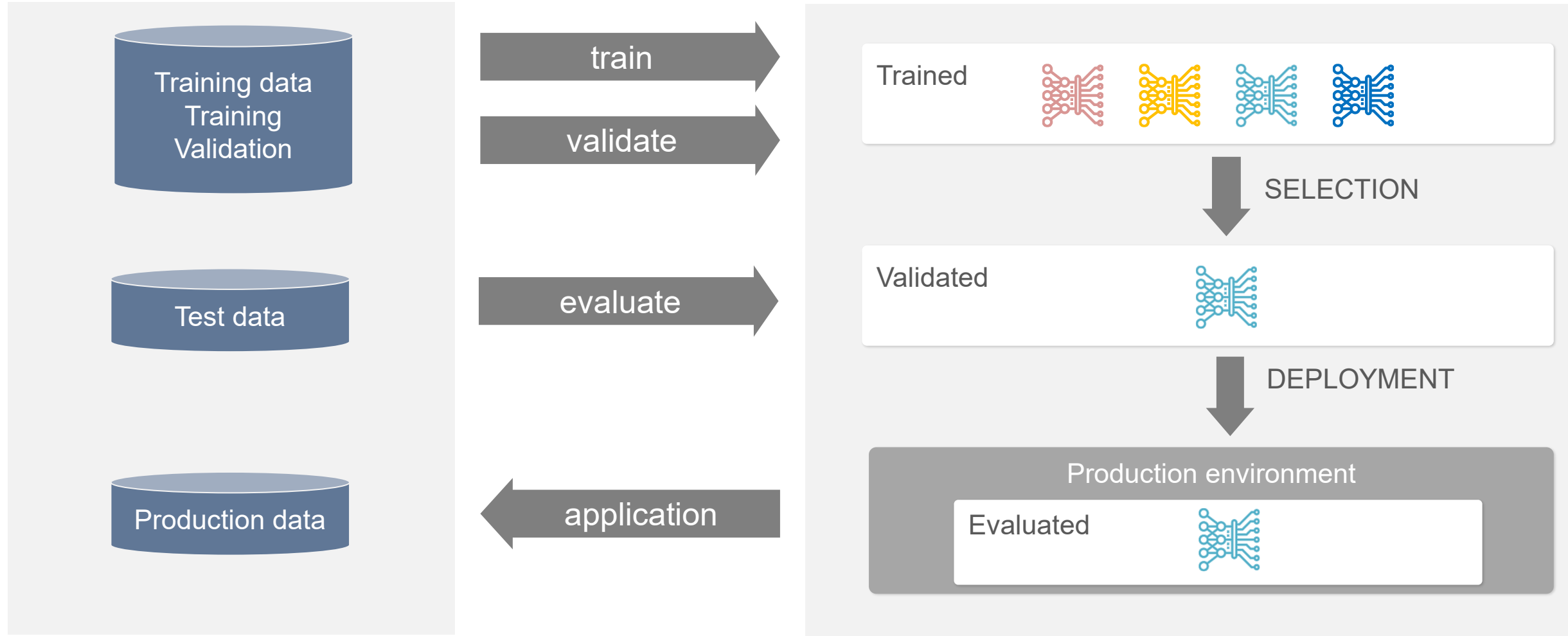


DATA ARCHITECT

Defines and develops technical components
Understands the technical complexity of IT systems

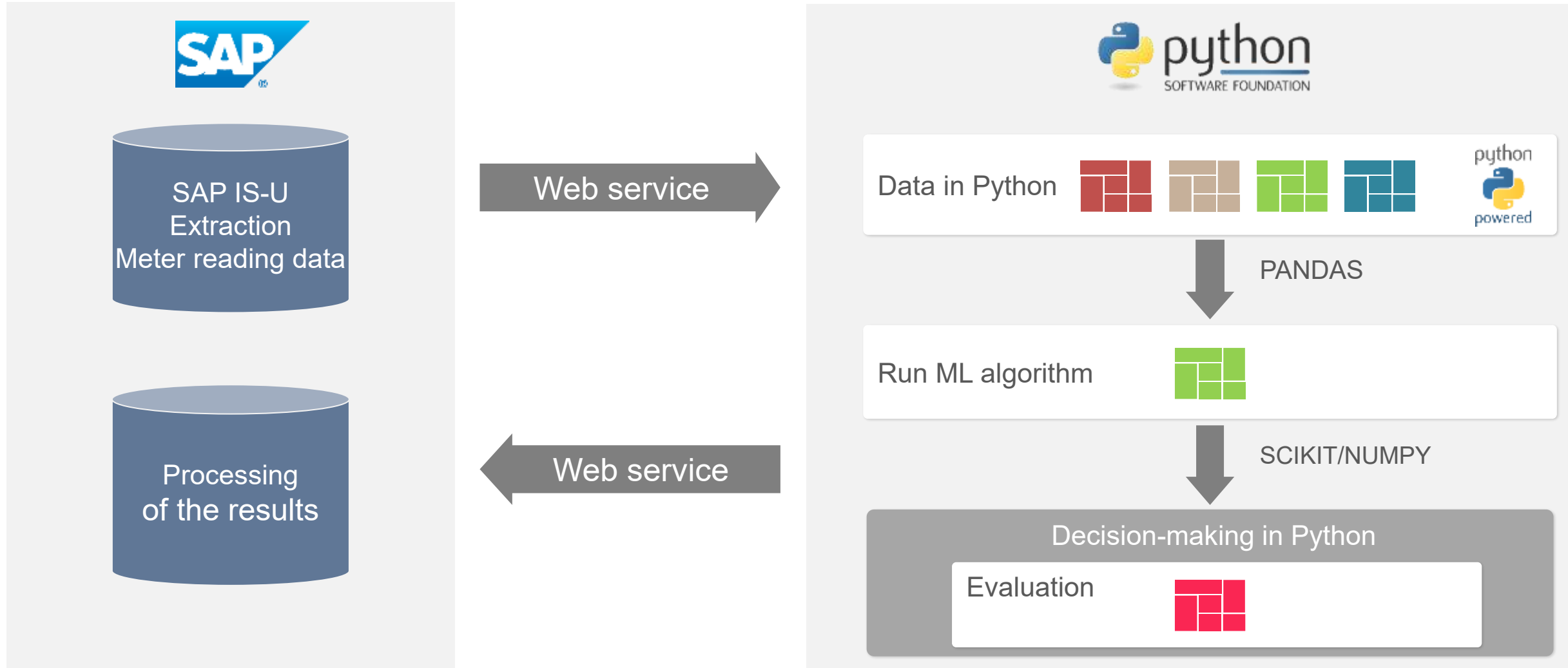
Source: DSC GmbH

IMPLEMENTING AI: KNOWLEDGE ACQUISITION AND MODELING



Source: Introduction to Data Science (Ch04), Prof. Dr. Bernhardt Drabant, Duale Hochschule Mannheim

IMPLEMENTING AI: IT ARCHITECTURE – SAP AND PYTHON



Source: DSC GmbH

THE END!



Please refer any questions to:
Prof. Dr. Florian Kauffeldt
Faculty of International Business
florian.kauffeldt@hs-heilbronn.de